

Design and Implementation of an Acoustic QPSK Transceiver

Mirudhula J - EE23B046

Chandrika K - EE23B033

1 Introduction

This project focuses on the implementation of an acoustic wireless communication system using Quadrature Phase Shift Keying (QPSK). Binary data is transmitted through air using speakers and microphones as the communication medium.

The system includes:

- QPSK modulation and demodulation
- Zadoff-Chu preamble synchronization
- Root Raised Cosine pulse shaping
- Upconversion and downconversion
- Carrier Frequency Offset (CFO) compensation
- PLL phase recovery
- Gardner timing recovery
- BER analysis

The complete system was first verified in simulation using AWGN, phase offsets, timing offsets, and CFO impairments before attempting real acoustic transmission.

2 System Overview

The communication chain is:

Bits $\rightarrow$ QPSK Mapper $\rightarrow$ ZC Preamble $\rightarrow$ RRC Filter $\rightarrow$ Upconversion $\rightarrow$ Acoustic Channel

Received Signal $\rightarrow$ Downconversion $\rightarrow$ Synchronization $\rightarrow$ CFO Compensation $\rightarrow$ Timing Recovery $\rightarrow$ Matched F

3 Transmitter

3.1 QPSK Mapping

Two bits are mapped into one QPSK symbol:

$$00 \rightarrow +1 + j$$

$$01 \rightarrow -1 + j$$

$$11 \rightarrow -1 - j$$

$$10 \rightarrow +1 - j$$

The complex baseband signal is:

$$s[n] = I[n] + jQ[n]$$

3.2 Zadoff-Chu Preamble

A Zadoff-Chu (ZC) sequence is used for synchronization:

$$x(n) = e^{-j\frac{\pi un(n+1)}{N}}$$

Advantages:

- Sharp autocorrelation peak
- Good synchronization
- Constant envelope

Frame structure:

$$\text{Frame} = \text{Preamble} + \text{Payload}$$

3.3 Pulse Shaping and Upconversion

A Root Raised Cosine (RRC) filter reduces ISI and limits bandwidth.

The passband signal is generated as:

$$s(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t)$$

The signal is transmitted acoustically through speakers.

4 Receiver

4.1 Downconversion

The received signal is converted back to baseband:

$$I(t) = r(t) \cos(2\pi f_c t)$$

$$Q(t) = r(t) \sin(2\pi f_c t)$$

4.2 Frame Synchronization

Synchronization is performed using correlation with the known ZC preamble:

$$R[k] = \sum_n r[n] p^*[n - k]$$

The correlation peak determines the frame start.

4.3 Carrier Frequency Offset Compensation

CFO causes constellation rotation:

$$r[n] = s[n]e^{j2\pi\Delta f nT}$$

The preamble is used to estimate and compensate for CFO.

4.4 Timing Recovery

Timing recovery is implemented using the Gardner Loop:

$$e[n] = (x[nT + \frac{T}{2}] - x[nT - \frac{T}{2}])x[nT]$$

This stage proved difficult under real acoustic conditions due to multipath distortion and sampling mismatch.

4.5 Matched Filtering and BER

An RRC matched filter maximizes SNR and reduces ISI.

BER is computed as:

$$BER = \frac{\text{Bit Errors}}{\text{Total Bits}}$$

5 Simulation Results

The system was tested in simulation with:

- AWGN noise
- CFO
- Phase offsets
- Timing offsets

Simulation results showed successful synchronization and low BER under controlled conditions.

6 Challenges

Major practical challenges encountered were:

- CFO instability
- Timing synchronization failure
- Multipath reflections
- Environmental acoustic noise

The Gardner timing recovery loop could not consistently lock under real acoustic conditions.

7 Code

```
1 import numpy as np
2 import scipy.signal as sp
3
4 FS = 44100
5 FC = 5500
6 SPS = 4
7 ROLL = 0.35
8 RRC_TAPS = 81
9 LPF_TAPS = 81
10 ZC_LEN = 127
11 N_BITS = 100000
12
13 def make_rrc():
14     beta, sps, ntaps = ROLL, SPS, RRC_TAPS
15     t = np.arange(-ntaps//2, ntaps//2 + 1)/sps
16     h = np.zeros_like(t)
17
18     for i, ti in enumerate(t):
19         if abs(ti) < 1e-12:
20             h[i] = 1 - beta + 4*beta/np.pi
21         else:
22             h[i] = (
23                 np.sin(np.pi*ti*(1-beta)) +
24                 4*beta*ti*np.cos(np.pi*ti*(1+beta))
25             ) / (
26                 np.pi*ti*(1-(4*beta*ti)**2)
27             )
28
29     h /= np.sqrt(np.sum(h**2))
30     return h
31
32 RRC = make_rrc()
33
34 REFS = np.exp(
35     1j*np.array([1,3,5,7])*np.pi/4
36 )
37
38 def diff_encode(bits):
39
40     if len(bits)%2:
41         bits = np.append(bits,0)
42
43     pairs = bits.reshape(-1,2)
44
45     idx = np.zeros(len(pairs)+1,dtype=int)
46
47     enc = {
48         (0,0):0,
49         (0,1):1,
50         (1,1):2,
51         (1,0):3
52     }
53
54     for i,p in enumerate(pairs):
55         idx[i+1] = (idx[i]+enc[tuple(p)])%4
56
57     return REFS[idx]
58
59 def transmitter(bits):
60
61     syms = diff_encode(bits)
62
63     up = np.zeros(
64         len(syms)*SPS,
65         dtype=complex
66     )
67
68     up[::SPS] = syms
69
70     I = np.convolve(np.real(up),RRC)
71     Q = np.convolve(np.imag(up),RRC)
72
73     t = np.arange(len(I))/FS
74
```

```

75     tx = (
76         I*np.cos(2*np.pi*FC*t)
77         -
78         Q*np.sin(2*np.pi*FC*t)
79     )
80
81     return tx/np.max(np.abs(tx))
82
83 bits = np.random.randint(0,2,N_BITS)
84
85 tx = transmitter(bits)
86
87 print("Transmission Complete")

```

8 Output Plots

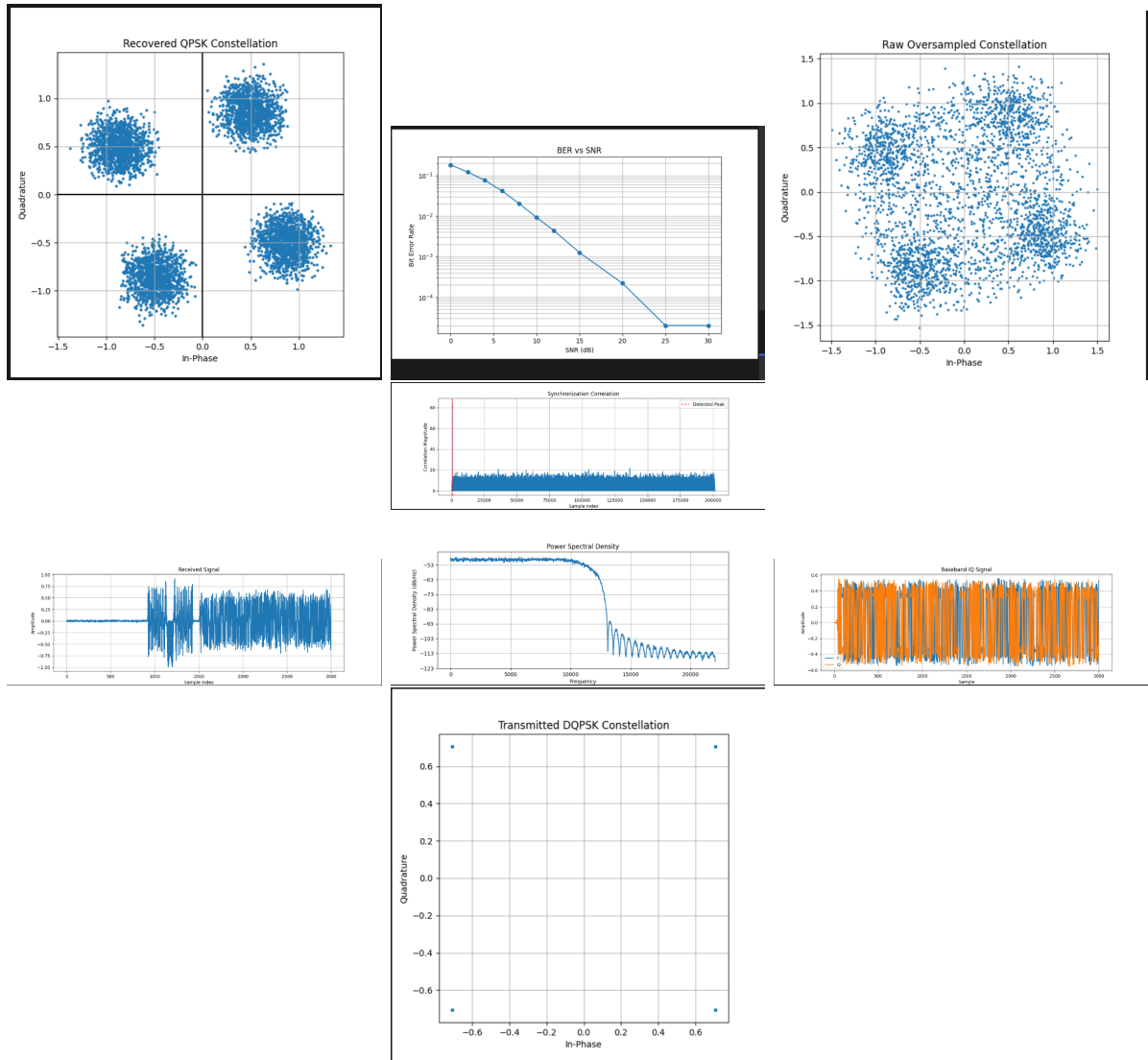


Figure 1: Simulation and Receiver Debugging Plots

9 Conclusion

An acoustic QPSK transceiver was successfully implemented and verified in simulation. The project demonstrated modulation, synchronization, pulse shaping, CFO correction, timing recovery, and BER analysis for wireless acoustic communication.

Although real-time acoustic synchronization remained challenging, the project provided significant

understanding of practical DSP implementation and synchronization issues in real communication systems.